

1. We are calculating the longest common subsequence (LCS) between two strings, $S = X_1X_2X_3X_4$ and $T = Y_1Y_2Y_3Y_4$. To do this, we fill an array C where C_{ij} represents the length of the LCS between the prefix of length i from S and the prefix of length j from T .

Some of the entries in the array C are currently masked using different symbols other than integers as shown below.

	Y_0	Y_1	Y_2	Y_3	Y_4
X_0	0	0	0	0	0
X_1	0	0	0	0	1
X_2	0	0	0	1	1
X_3	0	0	1	1	♡
X_4	0	1	♣	♠	◇

- i. What does the cell $C_{0,0}$ represent?
- ii. What can be said about the characters X_1 and Y_4 ?
- iii. What can be said about the characters X_2 and Y_4 ?
- iv. What is the value of ♡?
- v. What is the value of ♣?

2. The final exam has just ended, and you are on a well-deserved trip to a remote area. During your journey, you have collected different types of digital content that you want to share with your friends and family. Each *Content Type* has a *Total Duration* and a *Value*. Total Duration measures the content's pre-compression size, while Value reflects its importance or meaning to the audience.

Due to bandwidth constraints, one of your friends suggested using Huffman encoding, a lossless compression technique that assigns shorter binary codes to more voluminous (i.e., longer duration) items.

Content Type	Total Duration (Seconds)
A	13
B	11
C	9
D	8
E	6
F	5
G	4

- a) Construct a Huffman Tree based on the given **Total Duration** to assign optimal binary codes to each content type.
- b) Calculate the Total Number of Bits (*duration * code length*) required to transmit the full content of each type.

3. You are now attending the CSE221 final exam and there is a limited time to solve the questions. Each question requires a different amount of time and offers varying marks. After answering some questions, you suddenly realized that only 6 minutes remained. You aim to maximize the total marks you can score, given the constraints on time.

Questions and their properties:

- Question 4: Time = 4 minutes, Marks = 7
 - Question 5: Time = 2 minutes, Marks = 9
 - Question 6: Time = 3 minutes, Marks = 10
 - Question 7: Time = 5 minutes, Marks = 8
 - Question 8: Time = 6 minutes, Marks = 12
 - Question 9: Time = 1 minute, Marks = 3
- a) If your faculty doesn't give partial marks for incomplete answers, which combination of questions maximizes your total marks without exceeding the time limit? Hurry up, you need a quick, efficient approach—no time for brute force. **Show** a full detailed calculation.
- b) Fortunately, your faculty enters the hall and announces a new rule: partial marks will be given for partially completed answers. If you can solve part of a question, you'll get marks proportional to the time you spend on it. For example, solving half of a 4-minute question gives you half the marks.

Now, with this new opportunity, how does your strategy shift? How will you maximize your score within the time limit, factoring in that you can now earn something even if you don't fully complete a question? Calculate the maximum marks you can score with this fortunate announcement. Did you score more than the previous scenario?

4. DNAs contain genetic and hereditary information of almost every living organism. DNA Sequence Matching is a procedure to compare two DNA sequences and find similarity between them. Given below is the recurrence relation of a dynamic programming algorithm to perform sequence matching of two DNAs.

$$seq(i, j) = \max \begin{cases} seq(i, j-1) - 1, \\ seq(i-1, j) + p(i, j), \\ seq(i-1, j) - 1 \end{cases}$$

where,

$$\begin{aligned} seq(i, j) &= 0, & \text{if } i = 0 \text{ or } j = 0 \\ p(i, j) &= 1, & \text{if } DNA1[i] = DNA2[j] \\ p(i, j) &= -2, & \text{otherwise} \end{aligned}$$

As you can see, the formula takes two indices, i and j of two DNA sequences given as input. Then it produces a similarity score as output. We want to apply this formula to compare the following DNA sequences.

index	1	2	3	4	5
DNA1	G	C	G	T	A
DNA2	C	T	G	A	G

Now answer the following questions:

- Put appropriate values to fill the gaps in the line below:
 $seq(_, _) represents the final similarity score of the given DNAs.$
- Apply the formula to calculate a similarity score between the DNA sequences given above. Show your work with either a recursion tree or a memory table.
- State the time complexity of this algorithm with proper reasoning.
You can assume that, the lengths of the DNA sequences are N and M respectively.
- Mriaslun, a talented algorithmist, claims that he can implement the algorithm in an optimized way so that it only takes $O(\min(N, M))$ space. Can you do it too? Explain your idea.